Homework 2 Report: Distributed System with Multiple Virtual Machines

Course: Fundamentals of Distributed Computing

1. Overall System Architecture

The implemented system features a distributed, microservices-based architecture deployed across three independent virtual machines running Linux (Ubuntu). The main entry point for users is the Web Server machine (VM1), which acts solely as a gateway. This machine does not perform any local authentication nor does it store media files; instead, it communicates with the Authentication Server (VM2) to verify user logins and with the File Server (VM3) to retrieve images and files using the **JSON-RPC** protocol. Furthermore, a Pub/Sub architecture is integrated for memory monitoring. If the Web Server's RAM usage exceeds a predefined threshold, it publishes an alert event to a Subscriber.

2. VM Roles and IP Addresses

Based on the network configuration and implementation (as shown in the screenshots), the specifications for each machine are as follows:

- **VM1 (Web Server):** * **IP Address:** `10.142.247.80`
  - **Role:** Hosts HTML templates, receives login requests, acts as an RPC client to other servers, periodically monitors its own memory (Memory Monitor), and publishes events as a Publisher.
- **VM2 (Auth Server):**
  - **IP Address:** `10.142.247.174` (Running on port `8082`)
  - **Role:** Manages the `users.json` database and processes authentication logic. **(Design Bonus: Passwords are not stored in plain text; they are securely hashed using the SHA-256 algorithm).**
- **VM3 (File Server):**
  - **IP Address:** `10.142.247.99` (Running on port `8081`)
  - **Role:** Serves files and images. The Pub/Sub memory Subscriber server is also run independently on this machine (on port `8083`).

3. RPC Technology Analysis

This project utilizes **JSON-RPC** (using Go's standard `net/rpc/jsonrpc` package).

- **Reason for Selection:** Since our system is a lightweight web-based environment, JSON-RPC is a highly logical and efficient choice. It uses a human-readable JSON format (making debugging much easier), is simple to implement, and does not require complex interface definition language (IDL) setups or contract compilation (unlike gRPC with Protocol Buffers).
- **Implemented Procedures:**
    1. `AuthService.Login`: Executed on VM2. It receives a `LoginArgs` object (username and password), hashes the incoming password, compares it with the database, and returns a `LoginReply` object (success status and message).
    2. `FileService.GetFile`: Executed on VM3. It takes a file category and filename, securely reads the file, and returns the raw file data.

4. File Retrieval from VM3 (Design Bonus)

To retrieve files from the third service, instead of using a basic HTTP static file server, the system utilizes a unified **JSON-RPC** channel, which fulfills the project's **Design Bonus** requirement. The workflow is as follows: after a successful login, the browser requests an image from VM1. VM1 acts as an RPC client and sends a request containing the filename (`secret.jpg`) to VM3. VM3 reads the file and returns it over the network as a byte array (`[]byte`) to VM1. Additionally, to prevent security vulnerabilities like Path Traversal, the `filepath.Base` function is strictly utilized on the File Server to sanitize incoming requests.

5. Pub/Sub Mechanism and Memory Monitoring

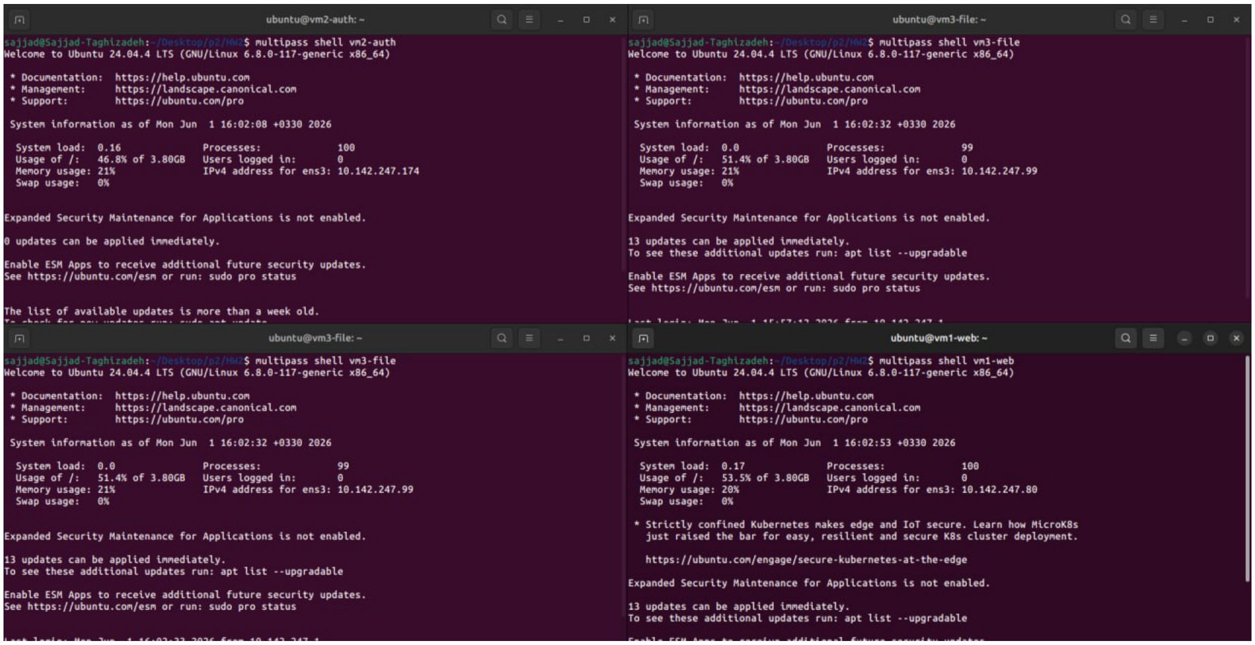
- **Memory Monitoring Method:** On the Web Server (VM1), a background Goroutine (`startMemoryMonitor`) is executed. Using a `Ticker`, it checks the allocated memory (Allocated Memory) every 3 seconds via the `runtime.ReadMemStats` command.
- **Pub/Sub Mechanism:** The memory usage threshold is set to **300 MB**. If memory consumption exceeds this limit, VM1 (acting as the Publisher) creates a JSON object containing event details (current memory, source service, and timestamp) and sends it via HTTP POST to the Subscriber's address. The Subscriber server receives this event and prints an `URGENT ALERT` message to the terminal.

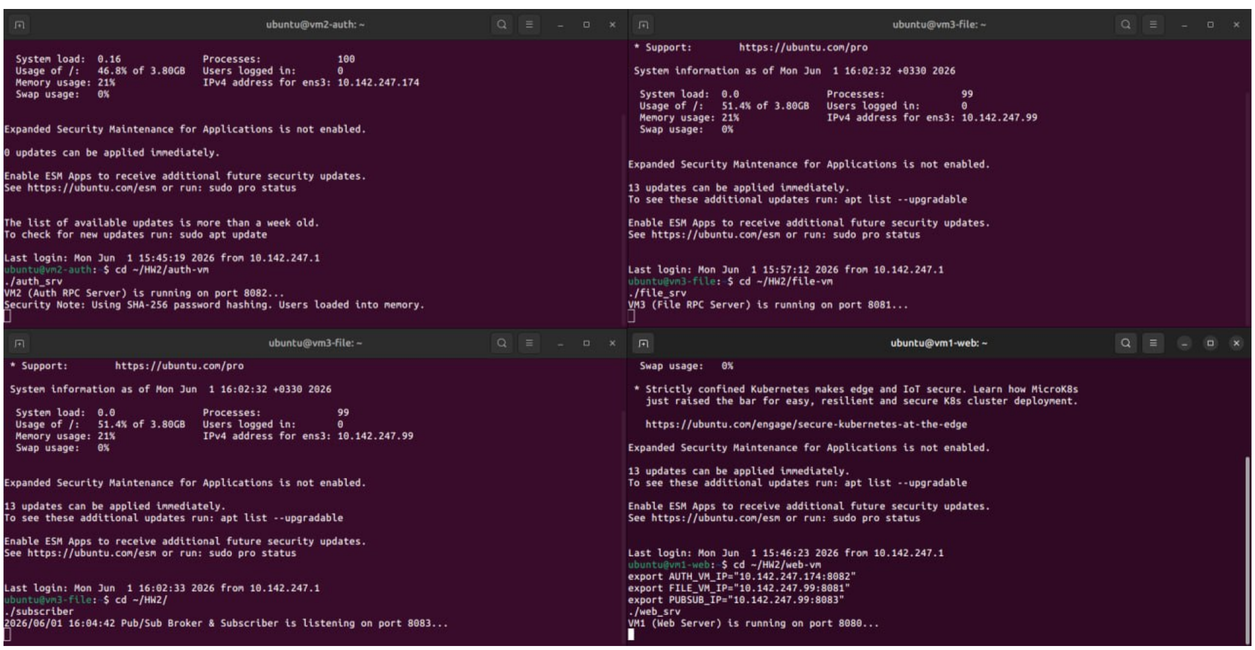
6. Testing Scenario and Execution Documentation

All testing steps were systematically executed in the Ubuntu environment. The visual documentation is as follows:

s

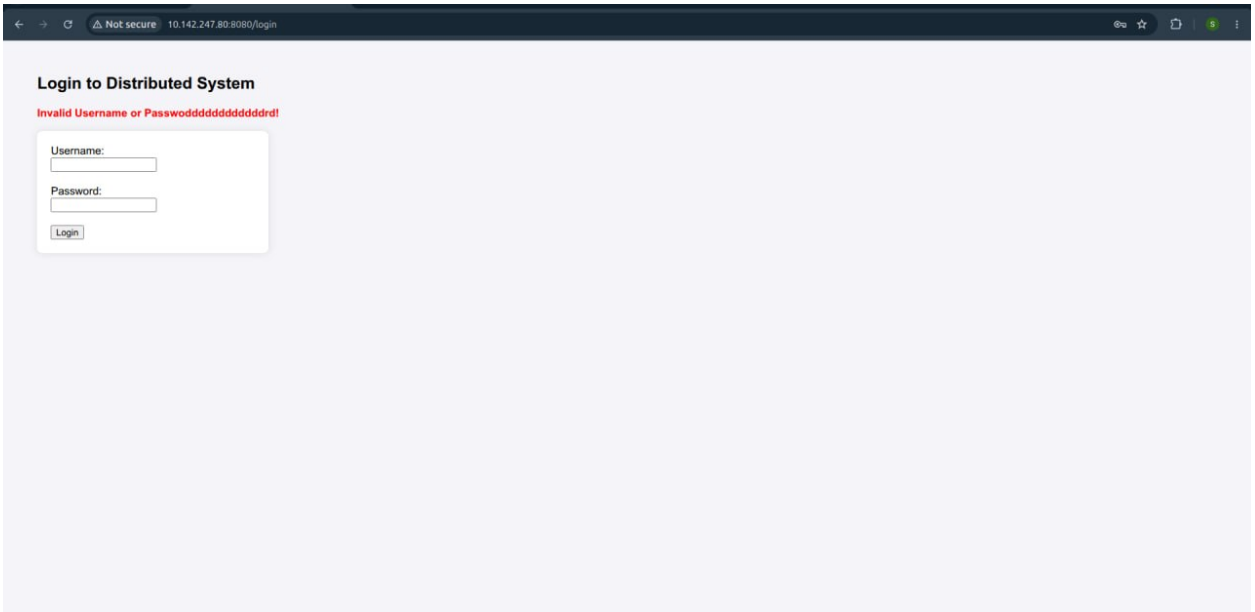
The following image shows the execution of the virtual machines and the startup logs of the RPC servers on their respective ports (8081, 8082, 8080, and 8083):





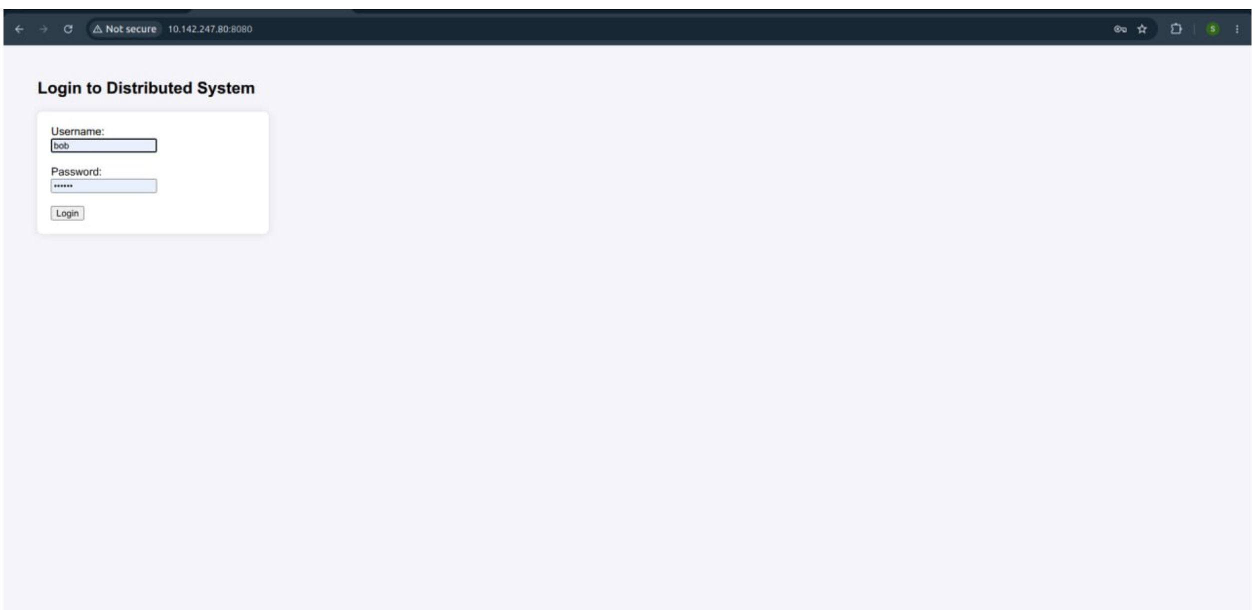
Step 2: Unsuccessful Login Test

Attempting to log in with incorrect credentials. The error is properly handled by VM2 and displayed on the web page:



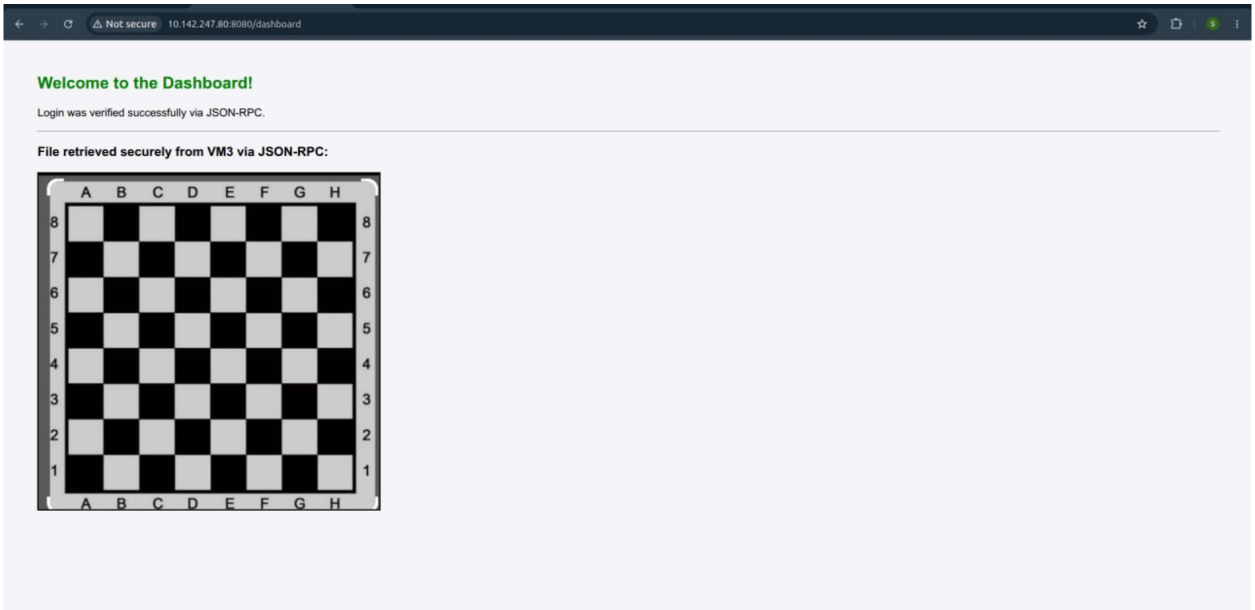
Step 3: Successful Login Test

Logging in with a valid user (`bob`). After verification by the Auth Server, the user is successfully redirected to the dashboard:



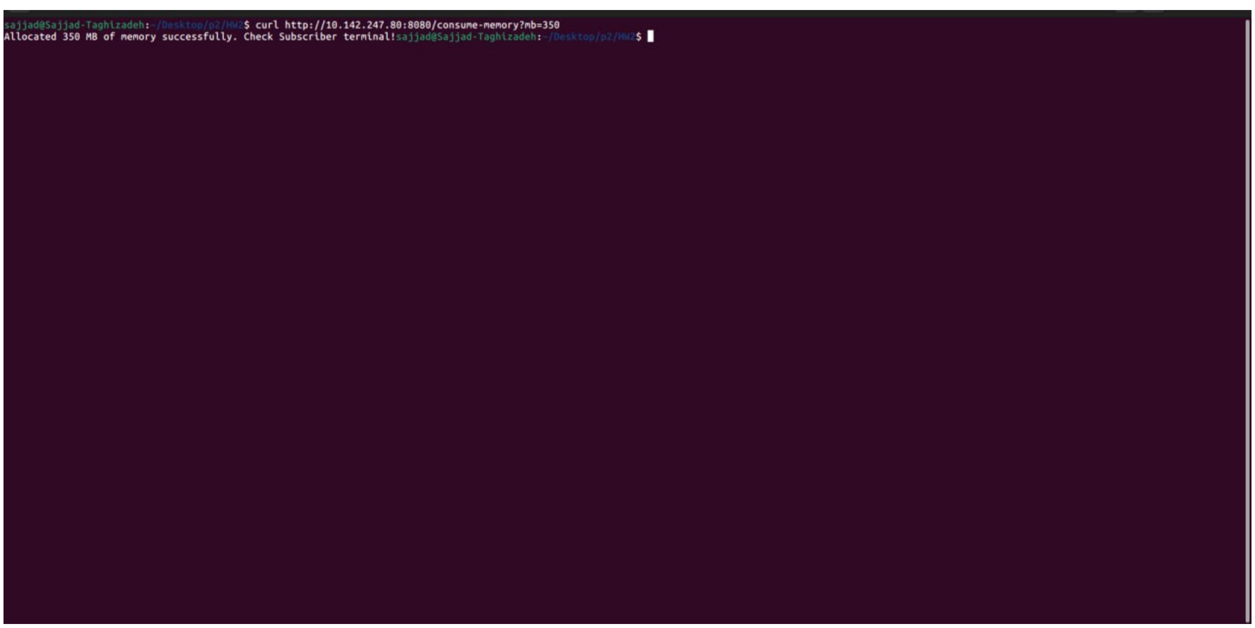
Step 4: Successful Image Retrieval from the File Machine

After logging in, the dashboard successfully displays an image (a chessboard) fetched from VM3 via JSON-RPC:

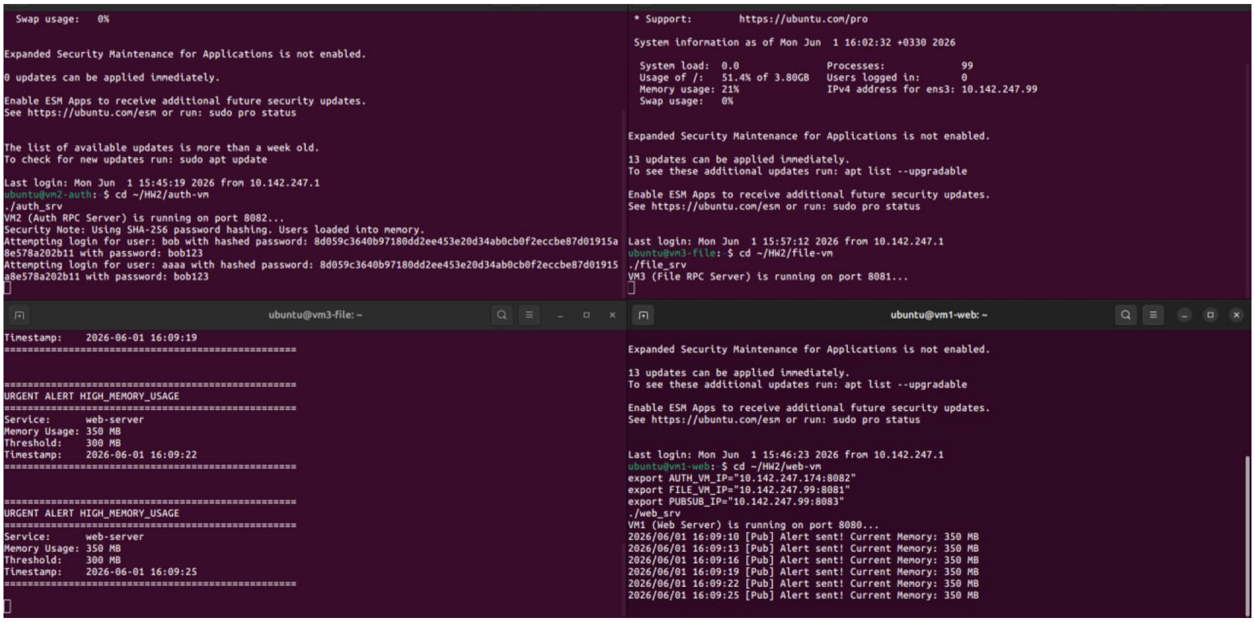


Step 5: Intentional Memory Allocation and Pub/Sub Alert

By executing a `curl` command to the `/consume-memory?mb=350` endpoint, the Web Server's memory is intentionally flooded.



Immediately after the memory consumption surpasses 300 MB, the monitoring system publishes the event, and the Subscriber prints the URGENT ALERT message along with detailed information in the terminal:



7. Implementation Challenges and Limitations

One of the main technical challenges in implementing the memory monitor was Go's automatic Garbage Collection (GC) behavior. If memory was allocated locally within a function, the GC would free it too rapidly for the monitor to detect the threshold breach. To solve this issue, a global slice variable (`memoryLeakSlice`) was defined. By storing the generated dummy data in this global heap structure, we successfully bypassed the automatic garbage cleanup, allowing us to simulate a genuine memory leak and successfully test the Pub/Sub alert system.